Design and Build a Balancing Robot

Name*[1]

[1]*Research School of Physics, Australian National University, Canberra ACT, 2601, Australia*

Abstract—

1. EXECUTIVE SUMMARY

Executive Summary

2. INTRODUCTION

Introduction

3. REQUIREMENTS REVIEW

The primary objective of this project is to build an autonomous self-balancing robot which is able to follow an designated path. To achieve this, the listed specific requirements are as follows:

- Top-level requirements:
  - The robot must maintain its balanced position without any falling cases.
  - If the robot is perturbed externally (e.g., a deliberated push, or an obstacle collided), it must be able to return balanced within $2s$. The perturbation is defined as changing its pitch angle up to 15°.
  - Without perturbation, the robot must balance itself in a small range. It must not move more than $0.5m$ in $1s$.
- Electronic components:
  - The robot must be able to sense its angle or velocity at $> 10Hz$. The resolution of angle must be $< 0.5°$.
  - The robot must take action within $0.1s$ after a controlling signal is sent.
  - The electronics must be stable during its runtime.
  - After turning on, the robot must work entirely independently, including power supply, signal processing, and controlling.
- Physical structure:
  - The structure must be stable enough that the position of mass center doesn't change.
  - The chassis must be sturdy, which can resist damage when the robot falls in some cases.

4. DESIGN DETAILS AND ANALYSIS

4.1. Control Logic

Fig. 1 shows the flowchart of the balancing logic of the robot. It's the basic logic, which aims to keep the robot balanced without any advanced action. It is based on the PID controller in the feedback control, using the output to tune its input until the robot is balanced. Code 1 shows the logical code written in Arduino IDE.

```
void setup() {
  Serial.begin(115200);
  setupSuccess &= connectImu();
  setupSuccess &= connectPID();
  setupSuccess &= connectMotor();
  Serial.print(F("Setup succeed: "));
  Serial.println(setupSuccess);
}

void loop() {
  if (!setupSuccess) { return; }
  if (!imuHasData) { return; }
```

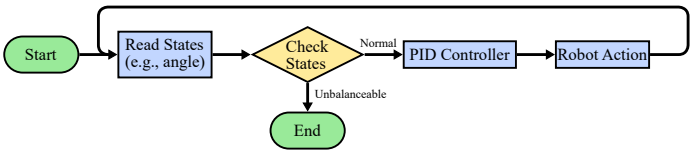


Figure 1. Flowchart of basic balancing logic. After turning on, the robot keeps reading its state, such as the pitch angle or the angular velocity of y-axis. If it's not strongly unbalanced, PID controller is used to generate an output from the state. The output is used to control the robot, such as moving forward or turn left. The feedback loop can balance the robot theoretically.

```
  if (!getImuAngle()) { return; }
  getMotorValue();
  driveMotor();
}
```

Code 1. Main logical code in Arduino IDE.

4.2. Electronic Components

To achieve the main goal, the electronic components corresponding to the control logic should contain these part: a state sensor, a central processor, an action driver, and power supply.

4.2.1. *The state sensor*

MPU-6050 inertial measurement unit (IMU)

4.2.2. *The central processor*

Arduino UNO

4.2.3. *The action driver*

L298N motor driver, Two motors and wheels

4.2.4. *Power supply*

Batteries, Breadboard

The electronic components used are:

- Arduino UNO
  - Pins: 14 Digital I/O pins, 6 PWM pins, 6 Analog input pins
  - Supported communications: I2C, SPI, UART
  - Power: Input voltage of 7-12V, I/O voltage of 5V
  -
- MPU-6050 inertial measurement unit (IMU)
- L298N motor driver
- Two motors and wheels
- Batteries
- Breadboard

Arduino UNO, MPU-6050 inertial measurement unit (IMU), L298N motor driver, motors, batteries and breadboard. Arduino UNO is the core of the robot, which runs the controlling program, process signals, and connects all the components together. We upload our control code ot it with a USB B connector. IMU is used to get the accelerations and angular velocities of all 3 axises, which can be further processed into other states including angles and velocities. The motor driver can receive analog signals from UNO using pulse width modulation (PWM), which controls the rotation speed of motors with variable values. The UNO and motor driver are powered by batteries with breadboard. The wiring method is shown in Fig. 2.

*email@anu.edu.au

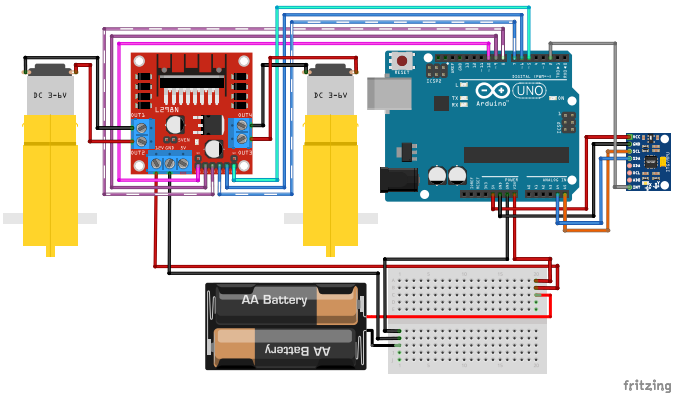


Figure 2. Wiring diagram of the electronic components. The battery and breadboard are used to power the Arduino UNO and L298N motor driver. UNO communicates with MPU-6050 inertial measurement unit (IMU) using I2C protocol, and sends analog signal to the motor driver using pulse width modulation (PWM). The motor driver controls 2 motors.

4.3. Signal Processing

4.3.1. IMU data processing

To get the robot state, we need to process the data from the IMU. The IMU contains a 3-axis gyroscope used to measure angular velocity and a 3-axis accelerometer used to measure acceleration. One important thing we can do is to get the angle (yaw, pitch, roll) from these values. Simply, we can integral the 3 angular velocities $(\omega_x, \omega_y, \omega_z)$ to get the corresponding Euler angle. But it suffers from Gimbal Lock (for example, the results are the same whether it yaws by 30° and then pitches by 90°, or pitches by 90° and then rolls by 30°, which makes it difficult to determine the true yaw or roll angle). Because of this, we use quaternion $q = (q_0, q_1, q_2, q_3)$ to represent the angle state, which indicates the current state that is equivalent to rotate the initial state around an axis (q_1, q_2, q_3) by an amount of q_0. The current quaternion is given by:

$$q_n = q_{n-1} + \frac{\Delta t}{2} \cdot q_{n-1} \otimes \omega \tag{1}$$

Another method to get quaternion is using the accelerations (a_x, a_y, a_z). Because there is a constant gravity of $\approx 9.8m \cdot s^{-2}$, we can know the quaternion by comparing the direction of measured acceleration with the true gravity. However, this method is noisy, while using gyroscope can produce drift over a long time. Therefore, we can use a weighted average of these two methods.

There are many states we can get from the quaternion. For example:

$$\text{Pitch: } \theta = \arcsin\left(2(q_0 q_2 - q_3 q_1)\right) \tag{2}$$

$$\text{Yaw: } \psi = \arctan \frac{2(q_0 q_3 + q_1 q_2)}{1 - 2(q_2^2 + q_3^2)} \tag{3}$$

$$\text{Acceleration: } a_{\text{world}} = q \otimes (0, a_x, a_y, a_z) \otimes q^{-1} \tag{4}$$

Most calculations mentioned above can be done by the IMU module or some libraries (e.g., `i2cdevlib/Arduino/MPU6050`)

4.3.2. PID controller

PID (Proportional-Integral-Derivative) controller is the core of the feedback system. By comparing the current state $S(t)$ with the desired state, we get an error term $e(t)$. Using $e(t)$ as the input of PID controller, we can use the output $u(t)$ to drive the current state toward the desired state. The output expression is given by:

$$u(t) = K_p e(t) + K_i \int_0^t d\tau\, e(\tau) + K_d \frac{de(t)}{dt} \tag{5}$$

The Proportional term means that the larger the error is, the large the output is to restore the state. The Integral term means that the longer the error persists, the large the output is to further restore the state. The Derivative term means that the faster the error decreases, the smaller the output is to reduce oscillation. The three terms have individual coefficients (K_p, K_i, K_d). Their combinations vary widely across different situations. By tunning these coefficients and looking for a precise balancing state, we can achieve better feedback performance.

4.3.3. Pulse width modulation

In order to control the robot smoothly and continuously, we need analog signals to do the task. However, the UNO has only digital output, which means the output voltage can be either HIGH level or LOW level without any values in between. PWM can solve this problem. Instead of encoding signals to voltage level, it produces a train of pulses as the output, and encodes signals to the width of each pulse. Meanwhile, our motor driver is able to receive PWM signals. In this way, there are much more available values to control.

4.4. Mechanical Design

4.4.1. Physical model analysis

Fig. 3 shows the simplified physical model of the robot. We can use Lagrangian mechanics to analyze the relationship between F and θ, x. The system Lagrangian and Euler–Lagrange equations are given by:

$$L = \frac{1}{2}M\dot{x}^2 + \frac{1}{2}m\left[(\dot{x} + l\dot{\theta}\cos\theta)^2 + (l\dot{\theta}\sin\theta)^2\right] - mgl\cos\theta \tag{6}$$

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{x}} - \frac{\partial L}{\partial x} = F \quad \Rightarrow \quad (M+m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = F \tag{7}$$

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}} - \frac{\partial L}{\partial \theta} = 0 \quad \Rightarrow \quad \ddot{x}\cos\theta + l\ddot{\theta} - g\sin\theta = 0 \tag{8}$$

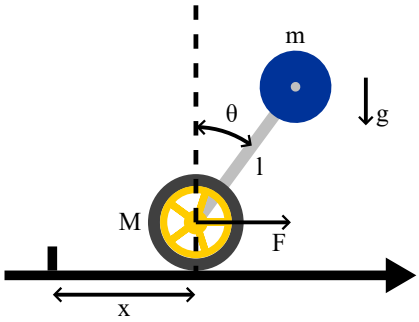


Figure 3. A simplified physical model of the robot. Constrain the movement in 1 dimension. THe model consists of a wheel with mass M, a body mass center m, and a light stick connecting these two with length l. The action of wheel is simplified as a force F. Assume the robot is at position x and its pitch angle is θ.

Usually, we care about the dynamic of θ, which can be derived from Equation 7 and Equation 8:

$$\ddot{\theta} = \frac{(M+m)g\sin\theta - F\cos\theta}{(M + m\sin^2\theta)l} - \frac{m\dot{\theta}^2\sin 2\theta}{2(M + m\sin^2\theta)} \tag{9}$$

According to the model Fig. 3 and this Equation 9, we can analyze some cases:

- The robot is perfectly at the balanced state, but a perturbation happens before the controller realizes it (i.e., $\theta \gtrapprox 0, F = \dot{\theta} = 0$). In this case, $\ddot{\theta} \approx \frac{(M+m)g\sin\theta}{lM}$. To save time for the controller to

realize before θ becomes too large, we want $\ddot{\theta}$ to be as small as possible, which means we need to use small m and/or large l.
- The robot is at an unbalanced but stable state (i.e., $\theta > 0$, $\ddot{\theta} \approx \dot{\theta} \approx 0$). To keep a constant θ, it's necessary to make $F = (M + m)g \tan\theta$. However, because of the limit of motor power, F can be maintained for only a short time before the large speed brings large resistance. So we should make $F > (M + m)g \tan\theta$ to bring back balance before the saturation of motors.
- The robot is severely unbalanced (i.e., $\theta > 0$, $\dot{\theta} > 0$). In this case, the only possible way to balance the robot is to make $\ddot{\theta}$ as negative as possible (i.e., $\ddot{\theta} \ll 0$). To achieve this, we need to use a combination of large $F > (M + m)g \tan\theta$ and small l.
- The robot is inverted (i.e., $90° < \theta < 270°$ ($l < R_{\text{wheel}}$), $l = 0$, or $m = 0$). Now the robot is impossible to return to the desired balanced state (or the balanced state can't be defined), but becomes permanently stable.

In conclusion, a motor being able to produce huge F is wanted for all the cases. Similarly, it's good to have small m, M, and/or $\frac{m}{M}$. Large l is suitable for peaceful environment and high balance performance, while small l or inverted robot is suitable for severe environment and high controllability.

4.4.2. *Chassis*

We use CAD (Autodesk Fusion) to design the chassis model, and 3D-print the model. The chassis is designed with many components in the experimental stage to make some components easier to update. These components are assembled using M3 bolts and nuts. For the final design, all the components are combined into a whole one to make the chassis stable.

In Fusion, the chassis is designed with the help of the models of other electronic components. Official datasheets of the electronic components are also used. In this way, it's precise and accurate to design the size and position of chassis components and through holes. One of our CAD model designs is shown in Fig. 4.

PrusaSlicer is used to convert CAD models to 3D-printing code. The printer uses PLA as the filament with diameter of 1.75mm. To make our chassis sturdy, we use layer height of 0.3mm. The vertical shells is set to 4 perimeters, and the horizontal shells is set to 4 top layers and 4 bottom layers. Since most of our chassis consist of plates of 3mm, these parameters are sufficient to make our chassis solid. The fill density is set to 15%.

4.5. Current Progress

Current Progress

5. PROJECT MANAGEMENT

5.1. Responsibilities

Responsibilities

5.2. Timeline

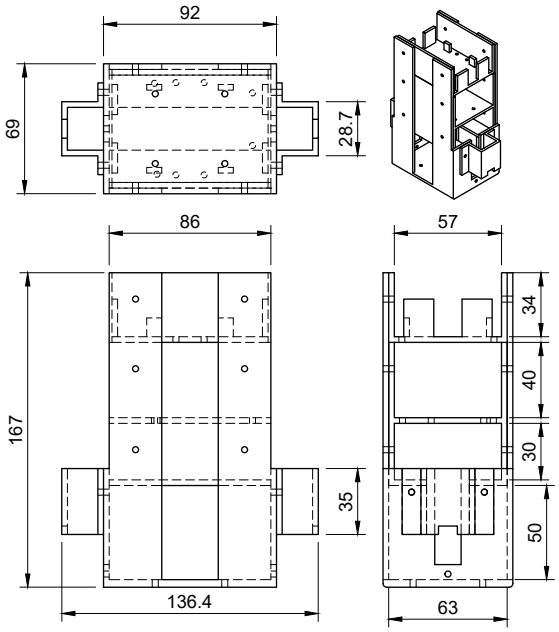
Timeline

5.3. Budget

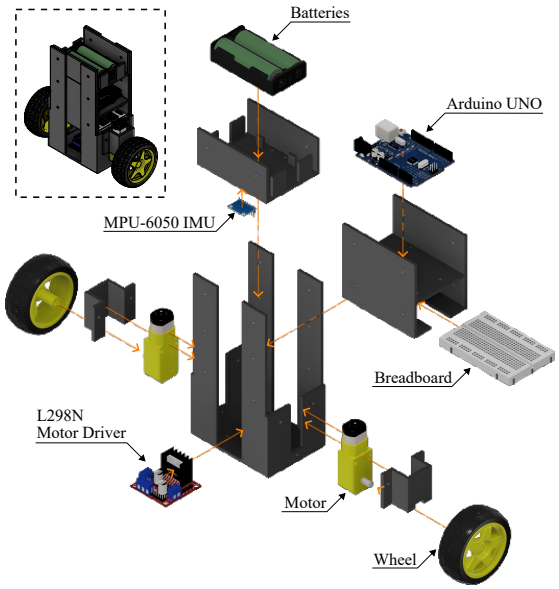
Budget

6. CONCLUSION

Conclusion



(a) Front-top-side view



(b) Robot components and assembly

Figure 4. A chassis model design.